

Hummingbird APIs

(Draft, last updated on 10.08.2026)

Overview

This document defines the APIs between the marketplace and the client, which purchases reservations, as well as between the marketplace and the AS, which offers reservations. It focuses on the **communication between these components**. The document does not specify the internal semantics of the marketplace, nor does it cover auxiliary elements such as configuration file formats. Support for the Sui-based marketplace can be enabled through a shim layer, as outlined towards the end of this document. **The document strictly follows the mechanisms outlined in the [Hummingbird paper](#).**

Reference Implementation

A proof-of-concept implementation of the communication and authentication mechanisms proposed in this document is available [here](#). It also provides an instance of a proof-of-concept marketplace implementation as well as a command-line interface for both users and AS operators, supporting user functions including asset listing, purchase, and redemption, as well as AS functions such as registration, asset publication and updates, statistics retrieval, redemption delegation, and password management.

Assumptions

- Although the marketplace primarily serves as an **orchestrator** that distributes bandwidth to clients, the API should also support both buying and selling bandwidth.
- The marketplace is trusted by both clients and ASes. In particular, the *centralized* marketplace is also trusted to secure access to the reservation authentication keys associated with redeemed assets.
- In this initial version, purchased assets are not necessarily transferable and are not guaranteed to be tradable further after acquisition.
- The marketplace maintains accounts for both ASes and clients.
- It stores purchased but unredeemed assets, as well as redeemed reservations, including the associated reservation metadata and authentication keys, under the respective client account.
- Each asset is uniquely identified by an asset ID within a given marketplace. No two assets share the same ID within the same marketplace.
- Each marketplace only supports a single currency.

Communication Method

Communication is implemented using **ConnectRPC** over HTTP/2.¹ For readability, message formats in this document are expressed using a JSON-like notation, while the actual wire format is defined using **Protocol Buffers**. The AS redemption service acts as the client and initiates all connections to the marketplace service. The marketplace does not initiate any connections toward the AS. The AS maintains a **bidirectional streaming RPC** to receive redemption events from the marketplace. This stream remains open for the lifetime of the connection and is used for continuous delivery of redemption requests. In addition, the AS uses a **unary RPC** to upload assets to the marketplace. This operation is request-response and does not require a persistent stream. If the connection is interrupted, the AS is responsible for re-establishing the stream and any in-flight communication with the marketplace.

Marketplace

Marketplace Attributes

The marketplace exposes general attributes that can be queried by clients and ASes:

```
{
  "api_major_version": 1,
  "api_minor_version": 0,
  "currency": "USD",
  "currency_exponent": 5, // making the currency 10^-5 USD
  "max_statistics_granularity": 1d,
  "asset_validity_max": 1y, // ASes can't upload assets that are valid in more than 1 year
  "pricing_strategy": "static seller-defined pricing", // alternatives: e.g., "dynamic_pricing"
  "transaction_fee_relative": "5%",
  "transaction_fee_absolute": "2300",
  "split_combine_fee_absolute": "10",
  "authenticator_rotation_fee_absolute": "1", // per updated reservation
  "supports_redemption_delegation": true,
  "delegation_hourly_fee": 1150, // per hour
}
```

These attributes provide basic information required for interacting with the marketplace, such as API compatibility and the currency used for pricing. Both ASes and clients are required to check these attributes before publishing or buying assets. No registration is required in order to fetch marketplace attributes.

¹Only the redemption service requires HTTP/2 or QUIC, as the asset redemption endpoint uses bidirectional streaming, which is not supported by HTTP/1.1. The clients and management service can use HTTP/1.1, HTTP/2, or QUIC.

AS ↔ Marketplace (Management Service)

Publishing Assets

Assets are published to the marketplace using the following message format:

```
{
  "if_id_ingress": 3, // If not specified, then this is an egress asset.
  "if_id_egress": 5, // If not specified, then this is an ingress asset. If both, ingress and
egress are specified, this is an interface-pair asset.
  "bandwidth": 1_000_000, // in kbps
  "bandwidth_min_redemption": 100, // in kbps. The minimum bandwidth required for an
asset to be eligible for redemption.
  "bandwidth_max_redemption": 10_000_000, // in kbps. The maximum bandwidth required
for an asset to be eligible for redemption.
  "router_only": false // true means forwarding guarantees are only provided for the router,
not the network.
  "starts_at_time": 1763718482, // in seconds
  "stops_at_time": 1763718492, // in seconds
  "time_granularity_redemption": 1, // in seconds. At the time of redemption, each
(split/combined) asset must be a multiple of this.
  "time_min_duration_redemption": 30, // in seconds. The minimum duration that the asset
must meet in order to be eligible for redemption.
  "time_max_duration_redemption": 3000000, // in seconds. The maximum duration that the
asset must meet in order to be eligible for redemption.
  "price": 1 // per kbps per second; currency determined by the marketplace attributes call
}
```

All fields that specify bounds (e.g., `bandwidth_min_redemption`) are enforced only at redemption time, as indicated by their name. These bounds may be temporarily violated during buying, splitting, and combining operations.² Note that there is no field specifying the AS, as the AS is authenticated anyway, so this field is implicitly given already and will be stored at the marketplace along with the asset. If a client needs more bandwidth than `bandwidth_max_redemption`, it needs to buy and redeem multiple reservations. For dynamic pricing, the `price` field represents the initial or base price, which may later be adjusted by the platform based on demand or other market conditions. If an AS wants to provide reserved bandwidth for reservations originating or terminating within that AS, it must explicitly publish ingress or egress assets with interface ID 0, or interface-pair assets of the form (0, X) or (X, 0), respectively.

² The reason for this can be explained through the following example. Consider two assets: some asset A covering the interval [0,100) and asset B covering [99,200), with `time_min_duration` = 2. Since the overlap between the two assets is only one second, they cannot be combined directly. Furthermore, enforcing the minimum-duration constraint during splitting would prevent B from being split in a way that resolves this issue. But if the constraint is enforced only at redemption time, B can instead be split into two assets: B → C: [99,100) and D: [100,200). Asset D can then be combined with A, resulting in a single asset covering [0,200). This flexibility allows intermediate operations to restructure assets while ensuring that all constraints are satisfied when the asset is ultimately redeemed.

When an AS uploads an asset, it commits to its contents. The AS may modify the asset as long as it has not been sold. Once a sub-asset is sold, however, it must no longer be possible for the AS to modify it. For example, if an AS later starts supporting intra-network forwarding guarantees (`router_only` = false), it should leave all previously uploaded assets unchanged and apply the corresponding flag only to newly uploaded assets.

If the request for publishing an asset is successful, the marketplace returns a unique asset identifier:

```
{
  "asset_id": 123456789
}
```

The `asset_id` uniquely identifies the asset within the marketplace. If the request fails, an error code is returned instead:

```
{
  "error_code": 1234,
  "error_string": "Marketplace does not support ..."
}
```

Modification and Removal

An AS can remove or modify any assets it owns on the marketplace:

```
{
  "Assets": [
    {"asset_id": 123456789,
     "remove": false,
     ... other asset fields as defined above; used only if remove = false ...},
    {...},
  ]
}
```

In some cases, parts of an asset may be sold just before the request is processed, which can invalidate the provided asset ID. When this occurs, the AS may need to re-list its assets and retry the remove or modify operation using the updated asset identifiers.

The marketplace response includes a list of asset ids that were successfully removed or modified, along with a separate list of errors for the assets that could not be processed.

Reservation Authenticator Rotation

If an AS' secret key used to compute reservation authenticators is compromised, the AS can generate a new secret key and start using it for reservation redemption and traffic verification at the border router. However, there may be active reservations whose authenticators were computed using the compromised key. These reservations must be updated so that their authenticators are computed using the new secret key. To perform this update, the AS can request the marketplace to rotate the authenticators of all active reservations.

```
{
  "rotation": "start", // other options: "status", "cancel"
}
```

If a start of cancel request fails, the marketplace returns an error response:

```
{
  "error_code": 1234,
  "error_string": "something went wrong"
}
```

Canceling an operation stops further processing but does not revert entries that have already been processed. For the status request the following response is returned:

```
{
  "state": "running", // other options: "idle", "completed", "failed", "canceled"
  "total": 1024,
  "processed": 128,
}
```

If redemption is delegated to the marketplace, the AS must first update the redemption delegation with the new secret key. Only after the delegation has been updated should the AS initiate the authenticator rotation. The marketplace then recomputes the authenticators for all active reservations using the updated secret key.

If redemption is not delegated to the marketplace, the AS must first ensure that its redemption service has been updated to use the new secret key. In this case, the marketplace cannot recompute the authenticators itself. Instead, it updates all active reservations by sending redemption requests to the AS's redemption service. The redemption service returns a new reservation authenticator computed using the updated AS secret key.

Once the operation is complete, users can fetch their reservations from the marketplace again. The previously issued reservation authenticators will have been replaced with new authenticators computed using the new secret key. The reservation IDs may also differ between the original and updated reservations. Because rotating the authenticators of all active reservations can be an expensive operation, the marketplace may charge the AS for performing it.

Asset Details

Asset Constraints

Asset Type

It is recommended that an AS use either per-interface assets or per-interface-pair assets, rather than combining both approaches. ~~An AS must choose either per-interface assets or per-interface-pair assets and should not use a combination of the two.~~ The redemption service can be configured to create reservation IDs that are unique either per ingress interface or per ingress-egress interface pair. The border routers must be configured consistently with the same choice.

Capacity

ASes must upload to the marketplace only as much bandwidth as they can guarantee to forward. For a link with capacity C , sufficient slack is required to (i) ensure that the packet buffer does not grow significantly over time, thereby increasing latency (**parameter T**), and (ii) account for worst-case errors in the encoding of dataplane bandwidth (**parameter E**). While (E) depends on the choice of dataplane encoding, we recommend reserving **at least 10%** of the link capacity as slack to account for T.

While an AS may, in principle, announce multiple assets that overlap in time for the same interface or interface pair, provided it can guarantee forwarding for all of them, it is recommended to avoid such overlapping announcements.

Reservation ID Space

The available memory on the border router determines the maximum number of reservation IDs that can be supported. The maximum number of reservation IDs is implicitly determined by the asset size and `bandwidth_min`. The resulting reservation ID range is from 0 to `maxIDs - 1`.

Asset Data Plane Encoding

Floating Point

TODO: Describe encoding as defined in the paper.

Logarithmic

An AS can decide to represent bandwidth using a logarithmic scale, such that each bandwidth point is a fixed multiple of the previous one, so spacing grows exponentially across the range. This gives fine resolution at low bandwidths and coarser resolution at high bandwidths, while **keeping the relative error bounded**. By always rounding up to the next available level, the worst-case over-allocation is always the same small percentage, independent of the absolute bandwidth. For example, with `bandwidth_min_redemption` = 100 kbps to `bandwidth_max_redemption` = 10 Gbps, split into 1024 levels, each step is approximately 1% larger than the previous one (= parameter E in section “Asset Constraints”). If a client buys 1 Mbps, it might be rounded to ~1.01 Mbps, and if it buys 1 Gbps, it might be rounded to ~1.01 Gbps.

```
min := bandwidth_min_redemption // kbps
max := bandwidth_max_redemption // kbps
nr_codepoints := 2^10 // 1024 (codepoint range = [0, nr_codepoints - 1])

step := (max / min)^(1 / (nr_codepoints - 1))
scale := (nr_codepoints - 1) / log(max / min)

to_bw(codepoint) int -> int:
  precondition: 0 <= codepoint < nr_codepoints
```

```

    bw_kbps := min * step^codepoint
    return round(bw_kbps)

to_codepoint(bw_kbps) int -> int:
    precondition: min <= bw_kbps <= max

    codepoint := ceil(log(bw_kbps / min) * scale)
    return codepoint

```

Due to the ceil function, encoding a bandwidth always maps it to a codepoint representing an equal or higher bandwidth, i.e., $bw \leq to_bw(to_codepoint(bw))$. Depending on the value of `bandwidth_min_redemption`, multiple codepoints may map to the same bandwidth after rounding (for example, when `bandwidth_min_redemption` is close to 0 kbps). To avoid wasting codepoints in this region, the encoding can start with a linear section and switch to logarithmic spacing once each logarithmic codepoint maps to a distinct bandwidth value. The optimal transition point, i.e., the lowest value for which all codepoints are distinct, can be computed once during initialization

Asset Statistics

An AS can request statistics for the revenue and bandwidth utilization. The AS provides a statistics start time, a statistics end time, and a step size in seconds, all with a granularity defined by the marketplace `max_statistics_granularity` attribute. The AS can provide optional filters for ingress and egress IDs:

```

{
  "if_id_ingress": 1,
  "if_id_egress": 2,
  "start": 100000000, // start of the statistics window
  "end": 1100000000, // end of the statistics window
  "step": 86400 // interval step size in seconds
}

```

If the request is successful, the marketplace computes the number of intervals using start, end and step and returns for each interval the revenue and bandwidth utilization during that interval. Revenue and bandwidth utilization are calculated based on sold assets, regardless of whether the assets have been redeemed or not. If an asset's validity period is only partially contained within the requested start and end times, only the portion that overlaps with the requested time range is taken into account.

```

{
  "Statistics": [
    {
      "revenue": 5000, // per kbps per second of sold assets; currency according to
marketplace attributes
      "bandwidth_utilization": 0.125
    },
    {
      "revenue": 2000,

```

```

        "bandwidth_utilization": 0.05
    }
]
}

```

If the request fails, an error code is returned instead:

```

{
    "error_code": 3,
    "error_string": "invalid window"
}

```

Optional Redemption Delegation

The marketplace can offer an optional redemption delegation mechanism to the AS, allowing the AS to delegate execution of the redemption service to the marketplace instead of running it locally. Whether this feature is supported is defined using the `supports_redemption_delegation` in the marketplace attributes call. In the request, the AS defines when delegation should expire, which range of reservation IDs should be handed out per interface pair (starting with ID 0), and the secret key to derive reservation keys:

```

{
    "expiration_time": 10000000000,
    "redemption_lead_time": 9000, // max. amount of time before the reservation start at
    which redemption is allowed
    "reservation_id_lower_bound": 100, // inclusive; per interface pair
    "reservation_id_upper_bound": 200, // exclusive; per interface pair
    "key": 0xcafecafecafecafecafecafecafe,
    "encoding_points": [(0, 100), (1, 103), (2, 106), ...]
}

```

The AS must provide a sufficiently large reservation ID interval. If the interval is too small to satisfy a redeem asset request, the request fails. If the AS plans to take over redemption after `expiration_time`, it must ensure that it does not allocate reservation IDs that overlap with the previously provided reservation ID interval. If the request succeeds, the marketplace returns the following message:

```

{
    "expiration_time": 10000000000
}

```

Here, `expiration_time` is either equal to the requested `expiration_time`, or earlier. If the request fails, an error code is returned instead:

```

{
    "error_code": 12,
    "error_string": "marketplace does not support redemption delegation"
}

```

To prolong / replace the current delegation request, the AS can send a new delegation request with updated parameters, which takes effect immediately. A redemption delegation can be removed by expiring it, for example by submitting a redemption delegation request with an expiration date in the past.

Management Service Configuration

TODO: Add a link to Fabio's tool for uploading assets, using the topology file as a starting point, and for retrieving and visualizing statistics on sold bandwidth and revenue.

AS ↔ Marketplace (Redemption Service)

Redemption

To redeem an asset, the marketplace contacts the seller AS using the following message format:

```
{
  "request_id": 9874356,
  "ingress_id": 1234,
  "egress_id": 1000,
  "bw": 1024, // in kbps
  "starts_at": "1763718482",
  "stops_at": "1763718492"
}
```

The start and stop values are interpreted as precise instants on the timeline, not a whole second-long interval. The request is synchronous, meaning it returns only after either completing the operation or reaching a timeout. If the redemption is successful, the AS returns a success message containing the reservation information and the authorization key (**ak**):

```
{
  "request_id": 9874356,
  "res_info": {
    "res_id": 15060,
    "bw_rounded": 6600, // in kbps
    "bw_dataplane_encoding", // custom encoding
  },
  "ak": "0x..."
}
```

The **request_id** must match the corresponding field in the request. This identifier is required to support streaming in ConnectRPC. The **res_info** field only includes the reservation identifier and the final bandwidth allocated by the AS, since all other parameters (ingress and egress interface identifiers, as well as the validity period) are already specified in the request. In particular, returning the final allocated bandwidth is necessary because it can slightly differ from the bandwidth requested during redemption. This is due to the fact that the requested bandwidth is encoded using a custom 10-bit representation carried in Hummingbird dataplane packets. The encoding algorithm is defined independently by each AS. When a Hummingbird packet later arrives at the AS border router, the router translates the encoded value into the corresponding actual bandwidth. To make this mapping explicit to the client, the redemption response also includes the **bw_rounded** field, which specifies the precise bandwidth represented by the dataplane encoding.

Example: a client may purchase 3.5 Mbps from the marketplace. The AS could encode this value as 23 in the `bw_dataplane_encoding` field, where 23 is an AS-specific encoding with no global meaning. The response would additionally include `bw_rounded` = 3.6 Mbps, indicating that 3.6 Mbps is the closest bandwidth value supported by the AS encoding scheme to the originally requested 3.5 Mbps.

If the redemption fails, the AS returns an error message:

```
{
  "request_id": 9874356,
  "error_code": 1234,
  "error_string": "The ingress ID does not exist"
}
```

The `error_string` field is intended for logging and user-facing notifications. The marketplace may re-issue the request in case of failure, for example after transient errors. Every error message contains an error code and a human-readable string.

Redemption Consistency Checks

The rationale for placing a redemption service within the AS is that it preserves full control over the redemption process. However, if the AS were to approve every redemption request unconditionally, the service would provide no meaningful additional value, as in that case the redemption could just as well be handled entirely by the marketplace. To make the mechanism useful, the redemption service can act as a verifier of marketplace behavior. It does so by tracking both (i) the assets submitted to the marketplace and (ii) the cumulative amount of bandwidth redeemed over time. It then ensures that, at any point in time, the total redeemed bandwidth does not exceed the total assets that were initially uploaded.

TODO: This feature seems rather difficult to implement, as it requires precise coordination between the asset management service and the redemption service.

Client ↔ Marketplace

Marketplace Address (in PCBs)

Clients need a mechanism to discover the marketplace(s) where a given AS offers its assets. To enable this, each AS advertises the addresses of its associated marketplaces **via PCBs**. The information is structured as follows:

```
{
  "hummingbird": [
    {
      "name": "SCIONCreditCardMarketplace",
      "api_protocol": "connectrpc / TLS / QUIC / SCION",
      "api_address": "[1-ff00:0:110,192.0.2.1]:80"
      "client_registration_website": "https://www.centralized.org"
    },
    {

```

```

    "name": "IPCreditCardMarketplace",
    "api_protocol": "connectrpc / TLS / TCP",
    "api_address": "185.123.224.2:80",
    "client_registration_website": "https://www.newmarketplace.org"
  },
  {
    "name": "SuiMarketplace",
    "api_protocol": "connectrpc / TCP",
    "api_address": "localhost:5896" // local shim, no TLS required
    "client_registration_website": "https://www.sui.io"
  }
]
}

```

An AS can sell its bandwidth on multiple marketplaces, but the listed assets must be independent, to maintain resource guarantees for purchases.

In the short term, all information is included in the “notes” field of the PCB. In the long term, the goal is to properly incorporate it into the PCB format specification.

Searching Assets

A client can query the marketplace for available assets, as well as assets it already owns. The **owned** field distinguishes between these two cases. The request format is as follows:

```

{
  "owned": false, // Must be non-empty
  "ia": "1-ffaa:0:1107",
  "if_id_ingress": 3,
  "if_id_egress": 5,
  "min_required_bw": 10,
  "starts_at_latest": 1763718482,
  "stops_at_earliest": 1763718492,
  "time_min_duration_upper_bound": ...,
  "time_max_duration_lower_bound": ...,
  "price": 100, // per kbps per second; currency determined by the marketplace attributes
  "page": 1, // pagination starting at 0
  "max_returned_assets": 100 // The marketplace can impose its own limit
}

```

In general, any field that is omitted is not considered when constructing the search filter. Redeemed assets (flyovers) are not included in this response. A separate request is used to retrieve them. If the query is successful, the marketplace returns a list of matching assets available at the time of the query:

```

{
  "assets": [
    {
      "asset_id": "8368",
      "ia": "1-ffaa:0:1107",
      "if_id_ingress": 3,

```

```

        "if_id_egress": 5,
        "bandwidth": 1024,
        "bandwidth_min": 100,
        "bandwidth_max": 100_000,
        "starts_at": 1763718482,
        "stops_at": 1763718492,
        "time_granularity": 1,
        "time_min_duration": 1,
        "price": 1 // per kbps per second; currency determined by the marketplace attributes
    }
    ...
]
}

```

The returned assets reflect availability at the time the query is processed. They may no longer be available by the time the response is received by the client. If the query fails, the marketplace returns an error message:

```

{
  "error_code": 1234,
  "error_string": "Validity not-after is too far in the future"
}

```

Buying Assets

To purchase assets, the client must specify the ID(s) of the asset(s) it wants to acquire, along with the exact bandwidth and time interval requested. A purchase may involve splitting existing assets. Assets resulting from a split operation are assigned new, unique IDs, and the original asset gets removed.³ The client requests all desired asset splits and purchases in a single message:

```

{
  "assets": [
    {
      "asset_id": 123456789,
      "starts_at_exactly": 1763718482,
      "stops_at_exactly": 1763718485,
      "bw_exact": 256 // in kbps
    },
    {
      "asset_id": 666,
      "starts_at_exactly": 1763718485,
      "stops_at_exactly": 1763718495,
      "bw_exact": 256 // in kbps
    }
  ],
}

```

³ To prevent performance issues when many clients attempt to purchase portions of the same asset—and would otherwise need to repeatedly rediscover it and retry purchases because the asset ID changes after each split—the marketplace can batch client requests, enabling multiple purchases of a single asset to be processed together.

```

    "max_price": 10 // per kbps per second; currency according to marketplace attributes
}

```

If the request succeeds, the client receives the IDs of the acquired (possibly split) assets, in the same order as the request:

```

{
  "assets": [
    {
      "asset_id": 538575
    },
    {
      "asset_id": 963
    }
  ],
  "cost": 7 // currency determined by the marketplace attributes call
}

```

The purchased assets are then associated with the client's account in the marketplace. If the operation fails, no changes are applied (i.e., neither splitting nor purchasing is performed), and an error message is returned:

```

{
  "error_code": 1234,
  "error_string": "Insufficient credit"
}

```

Redemption

A redemption request has the following format:

```

{
  "ingress_asset_id": 123456789,
  "egress_asset_id": 5600,
  "if_pair_asset_id": // if specified, no ingress and egress asset ids must be specified
}

```

The call is synchronous and blocks until the operation either completes or times out. The response is either a success or an error message. When redeeming a reservation, a redemption request for the first AS on the path (i.e., the AS containing the source) can either use an interface-pair asset of the form (0, X) or specify an ingress asset with interface ID 0 together with an egress asset for the selected outgoing interface. Likewise, a redemption request for the last AS on the path (i.e., the AS containing the destination) can either use an interface-pair asset of the form (X, 0) or specify an ingress asset for the selected incoming interface together with an egress asset with interface ID 0.

If the redemption succeeds, the marketplace returns the authentication key (**ak**) and the reservation identifier, plus the reservation bandwidth:

```

{
  "ak": "0x...",
  "res_id": "123"
  "bw_rounded": 6600, // in kbps
  "bw_dataplane_encoding": // custom encoding
}

```

```
}
```

All information about redeemed reservations is also stored in the client's account.

In case of error, a corresponding message is returned:

```
{
  "error_code": 1234,
  "error_string": "The transit AS 1-ffaa:0:1107 responded: 'The ingress ID does not exist'"
}
```

If the request fails, no redemption is performed and no assets are removed from the marketplace.

Fetching Reservations

In case the client loses local state (e.g., due to a crash), it can retrieve its redeemed reservations using a search request:

```
{
  "ia": "1-ffaa:0:1107",
  "ingress_id": 1234,
  "egress_id": 1000,
  "bw": 10,
  "starts_at": 1763718482,
  "stops_at": 1763718492
}
```

Any field that is omitted is ignored when constructing the search filter.

If the query succeeds, the marketplace returns the matching reservations associated with the client:

```
{
  "reservations": [
    {
      "res_id": "123",
      "ia": "1-ffaa:0:1107",
      "ingress_id": 1234,
      "egress_id": 1000,
      "bw": 1024,
      "dataplane_encoding": 35,
      "starts_at": 1763718482,
      "stops_at": 1763718492,
      "ak": "0x..."
    }
    // ...
  ]
}
```

If the query fails, the marketplace returns an error message:

```
{
  "error_code": 1234,
  "error_string": "Invalid query parameters"
}
```

Splitting Assets

A user can split an asset they own into two or more new assets, either on the bandwidth axis or the time axis. The attributes of the new assets are inherited from the original asset. A split request must specify exactly one mode: either `bw_split` or `time_split`, but not both.

```
{
  "asset_id": 3,
  "bw_splits": [1763718487, ...] // The assets are assigned the specified portions of the
  original bandwidth. If the remainder is non-zero, a new asset is created and assigned to it.
  "time_splits": [1000000, ...] // The first asset is valid from the original start time up to the
  first specified split point (absolute timestamp), while the last asset is valid from the last
  specified split point until the original end time.
}
```

If the request is successful, the marketplace returns the newly created asset identifiers:

```
{
  "asset_ids": [72, 73, ...]
}
```

If the request fails, an error code is returned instead:

```
{
  "error_code": 3,
  "error_string": "invalid split request"
}
```

Combining Assets

A user may combine two or more assets they own into a single asset, provided they are compatible. To be eligible for combination, all assets must:

- Belong to the same IA and share the same ingress and egress with the same asset type.
- Asset attributes are combined as follows:
 - `min_bandwidth_redemption`: $\max(A, B, \dots)$
 - `max_bandwidth_redemption`: $\min(A, B, \dots)$
 - `time_min_duration_redemption`: $\max(A, B, \dots)$
 - `time_max_duration_redemption`: $\min(A, B, \dots)$
 - `time_granularity_redemption`: $\text{lcm}(A, B, \dots)$
 - `router_only`: $\text{or}(A, B, \dots)$

In addition, one of the following conditions must be met:

- The assets have adjacent validity periods and identical bandwidth → validity periods will be added together.
- The assets have identical validity periods (bandwidth may differ) → bandwidths will be added together.

A combine request specifies the identifiers of the assets to be merged:

```
{
  "assets": [72, 73]
```

```
}
```

If the operation succeeds, the system returns a new asset identifier representing the merged asset:

```
{
```

```
  "asset_id": 74,
```

```
}
```

If the operation fails, an error response is returned instead:

```
{
```

```
  "error_code": 3,
```

```
  "error_string": "assets cannot be combined"
```

```
}
```

Authentication and Confidentiality

For each operation defined in this API, both clients and ASes must authenticate themselves to the marketplace. Authentication is performed using a JSON Web Token (JWT) included in the gRPC metadata of each request. The marketplace authenticates itself to clients and ASes using TLS certificates. TLS also provides confidentiality and integrity for all communication.

The communication stack can be summarized as follows:

Application (ConnectRPC service logic)

↑

HTTP headers (JWT for client/AS authentication)

↑

HTTP/2 (ConnectRPC protocol)

↑

TLS (marketplace authentication via certificate, encryption in both directions)

The marketplace can use any certificate trusted by the operating system's root certificate store, for example one issued by Let's Encrypt. This applies to both TCP and SCION connections. In SCION, the client must provide the correct server name to enable proper hostname verification. For local testing, certificate verification can be disabled, or a certificate can be generated using a local certificate authority and installed into the OS root certificate store.

Per-application JWTs

Instead of issuing JWTs on a per-user basis, the marketplace can optionally offer JWTs at a finer granularity, such as the application level. This allows each application to independently purchase and redeem the appropriate assets. Consequently, an application cannot use reservations made by other applications or access their allocated monetary budgets.

Required JWT Tokens per Endpoint

The table below summarizes the JWT authentication requirements for each API endpoint. It specifies which token type, if any, is required to access and invoke the respective endpoint.

Endpoint	Required Token
Info	no token required
Publish Asset	Asset Publisher
Update Asset	Asset Publisher
Statistics	Asset Publisher
Search Assets	User, Asset Publisher
Buy Assets	User
Split Asset	User
Combine Assets	User
Fetch Reservations	User
Redeem Asset	User
Process Asset Redemption	Redemption Service
Delegate Redemption	Redemption Service
Reset JWT Token	User, Asset Publisher, Redemption Service
Set AS Password	Asset Publisher, Redemption Service

Account Creation

ASes and clients must have an account in the marketplace before they can interact with it.

AS Registration

ASes register in an automated way using their CP-PKI certificates to prove their identity via a challenge–response protocol. This approach has advantages over mTLS, particularly in deployments with reverse proxies that terminate TLS connections.

In this protocol, an AS initiates registration by sending its ISD-AS number. The marketplace responds by generating a random challenge ID and a nonce, which it stores together with the provided ISD-AS identifier. It then returns the challenge ID and nonce to the AS. The AS signs the nonce using its AS private key, including a marketplace-specific string as associated data (i.e., the HTTP host header sent by the AS and verified by the marketplace), and submits the challenge ID and the signed nonce. Upon receiving this information, the marketplace retrieves the corresponding challenge and verifies the signature using its cached trust material (TRCs). If the corresponding TRC is not available locally, it is fetched from the relevant AS. The marketplace verifies that the signature is valid, that it corresponds

to the claimed ISD-AS number, and that it matches the original challenge it issued. A cleanup routine periodically removes expired or uncompleted challenges. Once verification succeeds, the marketplace deletes the stored challenge and nonce, generates a JWT for managing assets at the marketplace and a separate JWT for the redemption service, and returns them to the AS.

Client Registration

Clients must also register an account before using the marketplace. The registration process for clients should include identity and potentially billing-related information. For example, payment credentials such as credit card information may be required, depending on the marketplace's operational model.

The user opens the marketplace website in a web browser and can either log in or register using a username and password. The user can then add the necessary marketplace-specific billing information and request JWT tokens by clicking a button on the website. These tokens can then be used to communicate with the marketplace API endpoints.

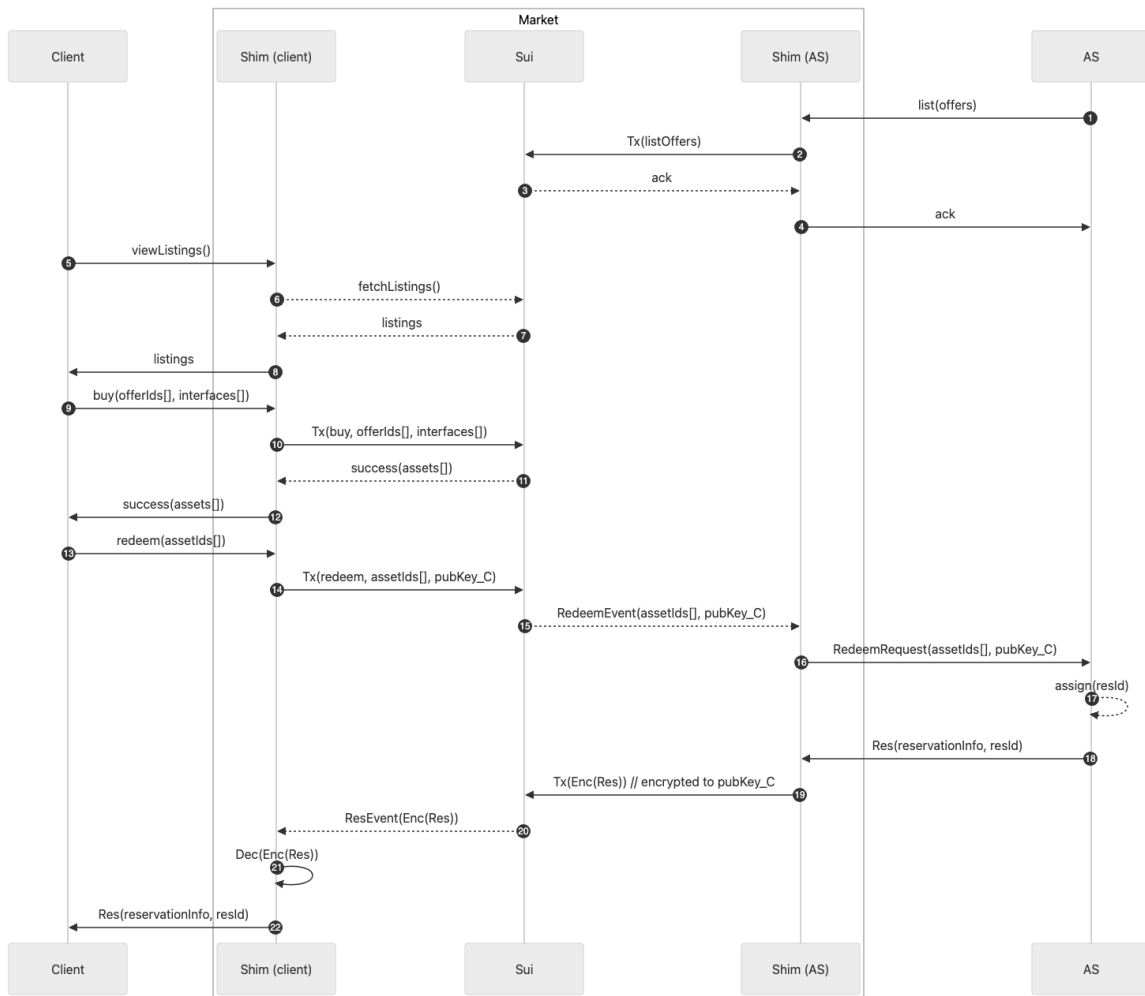
JWT Revocation

Each JWT carries a version claim (integer). The marketplace stores a per-user and per-AS active version. On authentication, the token is accepted only if its version matches the stored active version; otherwise it is rejected. Token issuance does not change the version. Revocation is performed by incrementing the stored version, which invalidates all issued tokens. To obtain new tokens, the AS must repeat the registration process, including presenting its certificate and successfully completing the challenge–response protocol using its private key.

Communication flow with Shim layers around Sui

To support a generic API that is compatible with both centralized marketplaces and the Sui-based marketplace, **local shim layers** are introduced at the client and AS. These shims handle interaction with Sui and manage wallet operations on behalf of the application. The sequence diagram below illustrates the overall flow. It is intentionally high-level; several parameters and auxiliary requests are omitted for clarity. In the diagram:

- Solid lines represent transactions and primary marketplace API calls.
- Dotted lines represent read-only queries to the blockchain state.



Reservation keys are not encrypted at the API level. Instead, encryption and decryption are handled by the client and the AS through the shim layer. This design reflects the distinction between:

- the public blockchain, where data is inherently visible, and
- centralized marketplaces, where sensitive reservation keys are only accessible to the user.

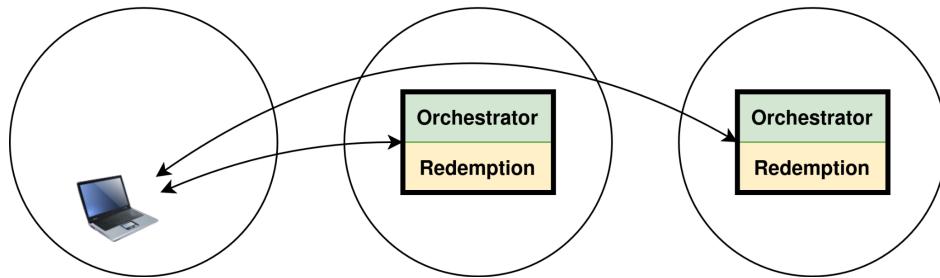
There are several important details in this design:

- The Sui shim does not require authentication. This is compatible with the API specification, as authentication is implemented as an additional layer and is not a strict requirement of the underlying gRPC interface.
- Gas fees are incurred when buying, splitting, or combining assets. The `max_price` field provided by the client does not include these fees. To prevent the client from being exposed to unexpectedly high gas costs, the client shim can potentially allow configuring an upper bound on the total gas fees the client is willing to pay.
- The shim can be extended to support additional features such as reservation caching, or even external backup to a cloud service.

Deployment

Minimal version

(Current deployment)



Advanced version (this document)

